

AI Chatbot với RAG: Từ Dữ Liệu Thô đến Câu Trả Lời Thông Minh

Phân tích chi tiết kiến trúc và luồng
xử lý của hệ thống

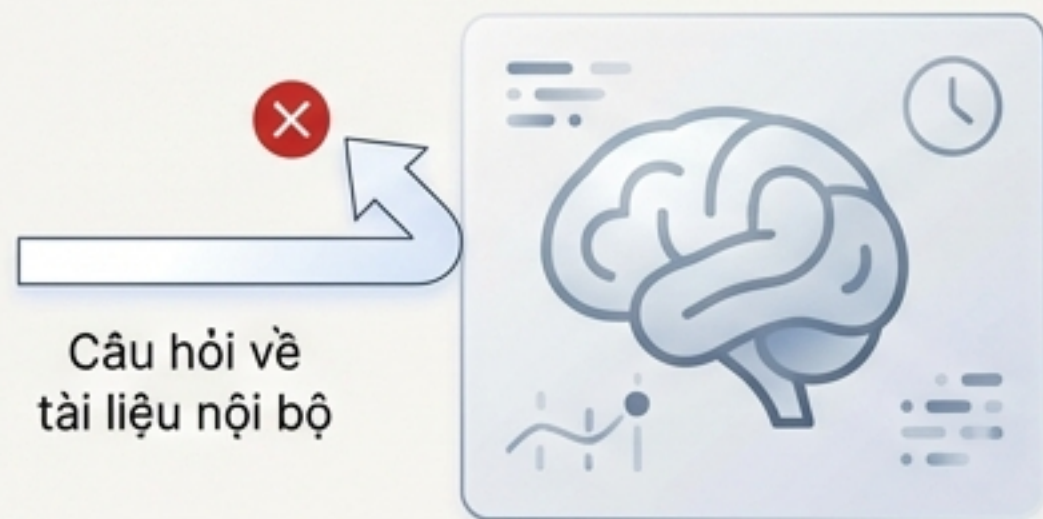
Tổng quan Kỹ thuật | 23 tháng 12, 2025

RAG giải quyết vấn đề "ảo giác" và giới hạn kiến thức của LLM truyền thống

Chatbot Thông Thường (LLM Only)



- Chỉ dựa vào kiến thức đã được huấn luyện (dữ liệu công khai, giới hạn thời gian).
- Không có kiến thức về tài liệu nội bộ, riêng tư của người dùng.
- Có nguy cơ cao bị "hallucination" (bịa đặt thông tin không có trong nguồn).



Chatbot với RAG

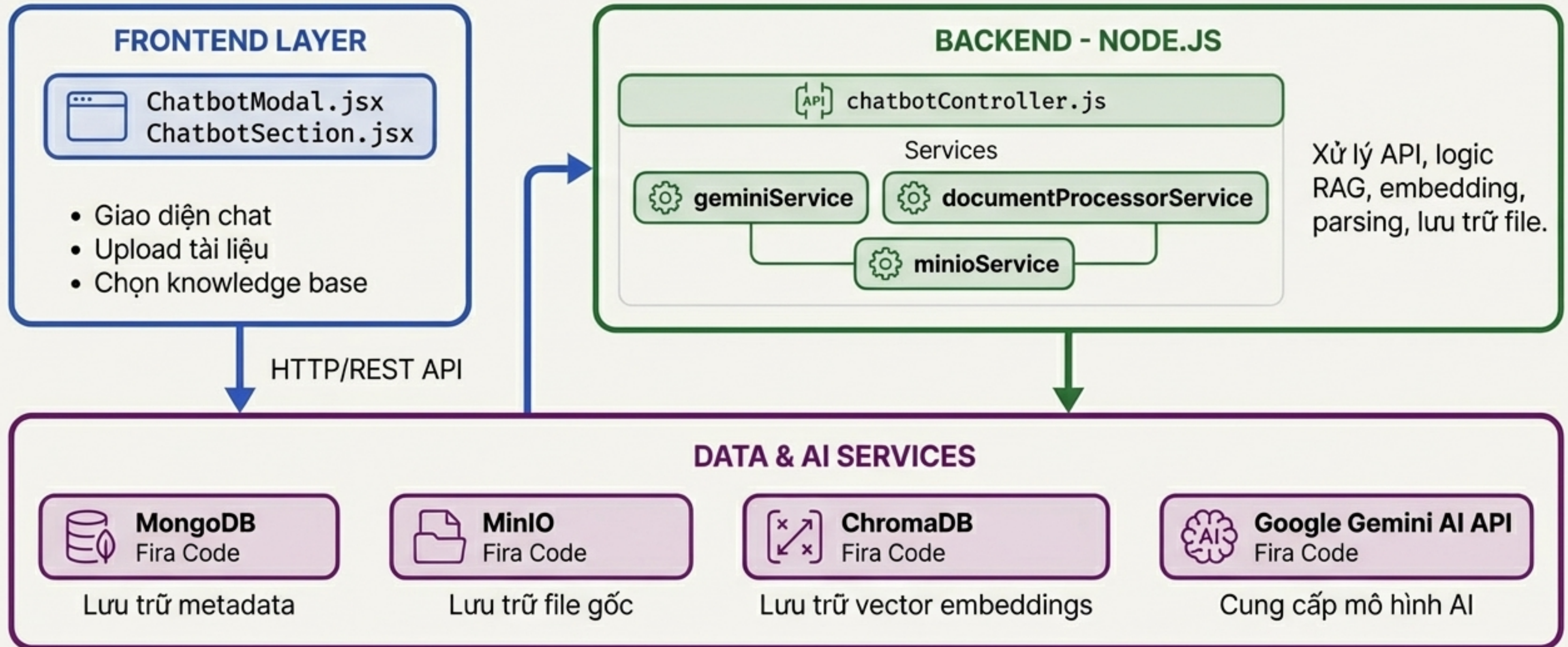


- Có khả năng truy vấn và sử dụng tài liệu riêng của người dùng (PDF, URL, Docs).
- Câu trả lời được sinh ra dựa trên nguồn thông tin thực tế, có thể kiểm chứng.
- Giảm thiểu đáng kể "hallucination" và tăng cường độ chính xác.



Sơ đồ kiến trúc tổng thể:

Hệ thống được cấu thành từ 3 lớp chính



Hành Trình 1: Biến Tài Liệu thành Tri Thức cho AI



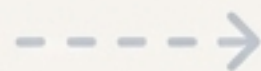
1. Tải Lên

Người dùng tải lên tài liệu (PDF, URL).

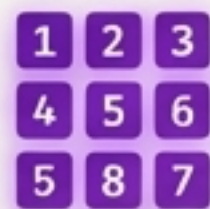


2. Phân Tích & Cắt Nhỏ

Hệ thống trích xuất văn bản và chia thành các đoạn nhỏ (chunks).



abc →



3. Vector Hóa

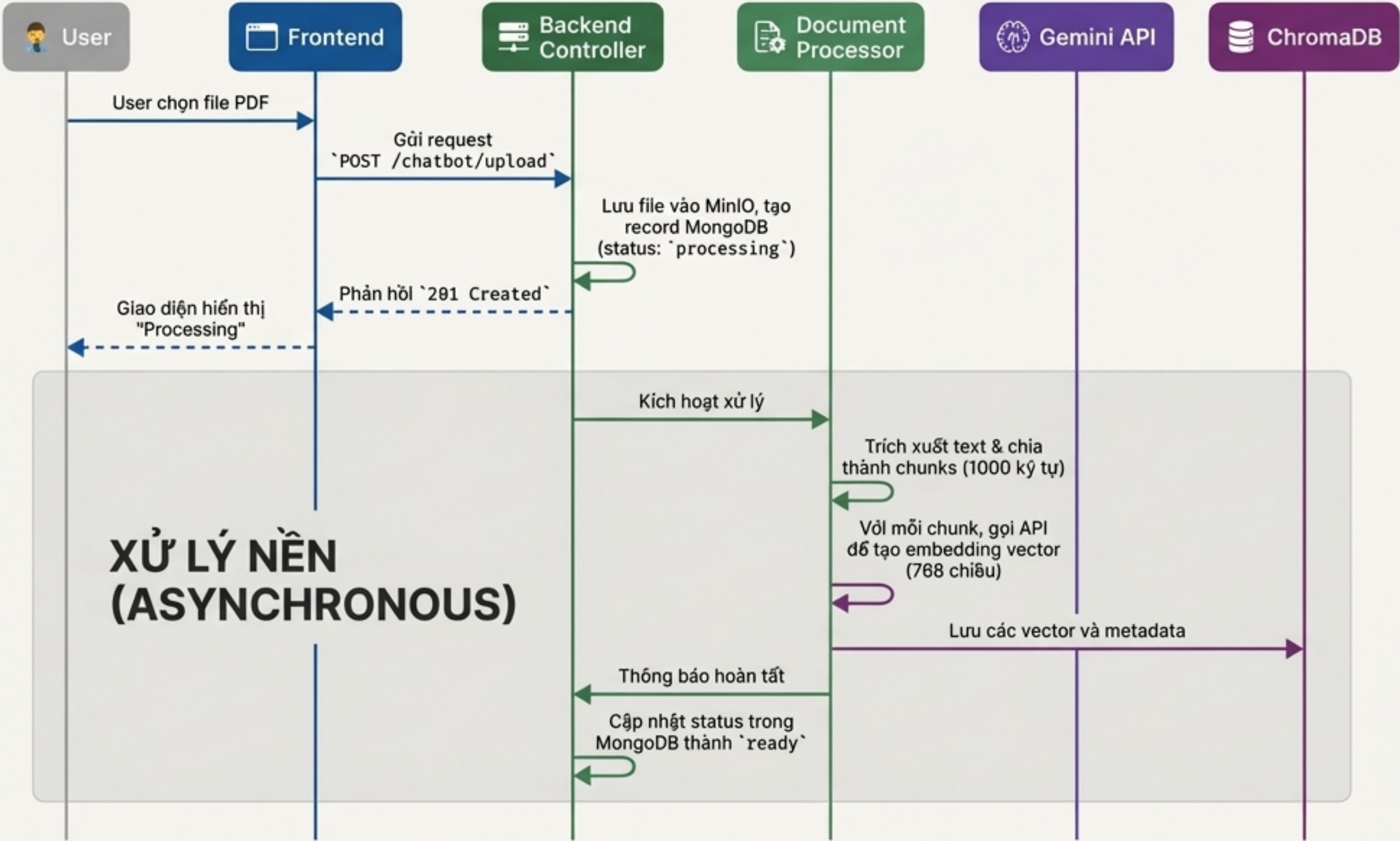
Mỗi chunk được chuyển thành một vector số học bằng ``text-embedding-004``.



4. Lưu Trữ

Các vector được lưu vào ChromaDB để sẵn sàng cho việc tìm kiếm.

Luồng xử lý chi tiết: Upload và lập chỉ mục tài liệu

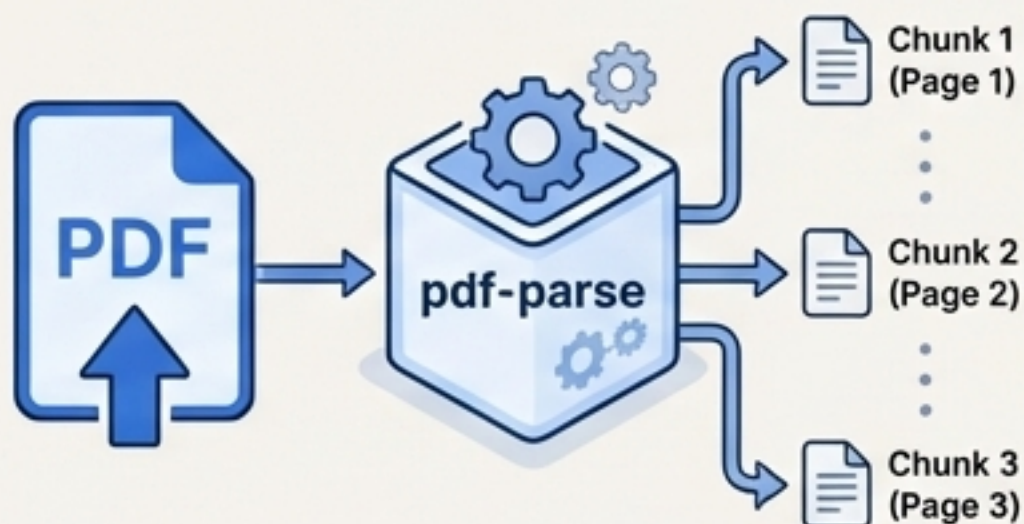


Phản hồi ngay lập tức cho người dùng trong khi các tác vụ nặng (parsing, embedding) được thực hiện trong nền giúp cải thiện trải nghiệm người dùng.

Cơ chế hoạt động của quá trình lập chỉ mục

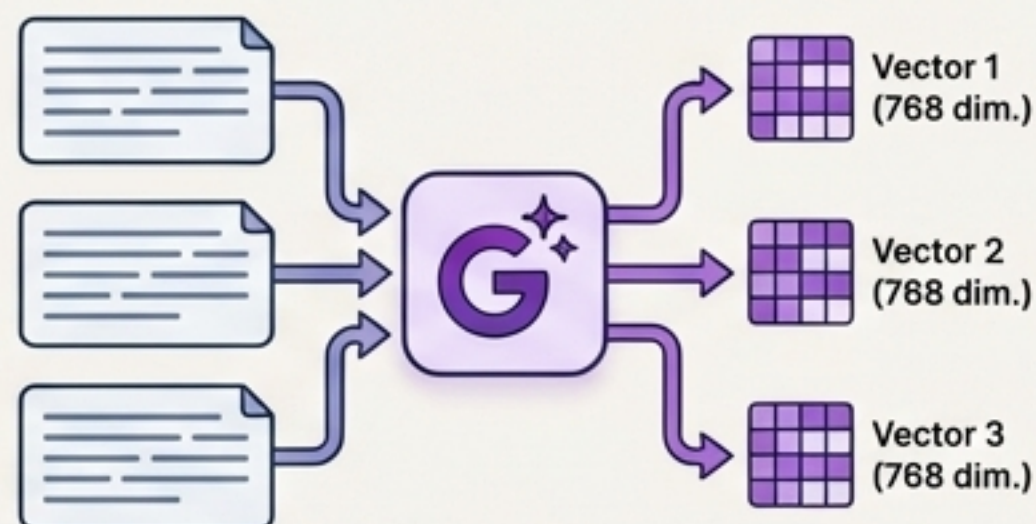
1. Phân tích & Cắt nhỏ (Parsing & Chunking)

- **Công cụ:** `pdf-parse`
- **Logic:** Trích xuất text từ PDF, sau đó chia thành các chunks có độ dài khoảng 1000 ký tự.
- **Metadata:** Mỗi chunk đều lưu giữ thông tin quan trọng như số trang (page number).



2. Tạo Embeddings

- **Mô hình:** `text-embedding-004` (Google Gemini)
- **Input:** Một chunk văn bản.
- **Output:** Một vector 768 chiều đại diện cho ngữ nghĩa của chunk đó.
- **Tối ưu:** Sử dụng xử lý song song (`Promise.all`) để tăng hiệu suất.



3. Lưu trữ Vector (Vector Storage)

- **Hệ quản trị:** ChromaDB
- **Cấu trúc:** Mỗi người dùng có một collection riêng (`kb_user_{userId}`).

```
{
  "id": "doc123_chunk_0",
  "embedding": [0.1, 0.2, ...],
  "document": "Văn bản gốc của chunk...",
  "metadata": {
    "documentId": "doc123",
    "chunkIndex": 0,
    "pageNumber": 1
  }
}
```


Hành Trình 2: Từ Câu Hỏi đến Câu Trả Lời Thông Minh



1. Câu Hỏi Người Dùng

Người dùng đặt câu hỏi và chọn các tài liệu liên quan.



2. Vector Hóa Câu Hỏi

Câu hỏi của người dùng được chuyển thành vector.



3. Tìm Kiếm

Tìm kiếm trong ChromaDB để lấy ra các chunk văn bản có vector gần nhất.



4. Xây Dựng Ngữ Cảnh

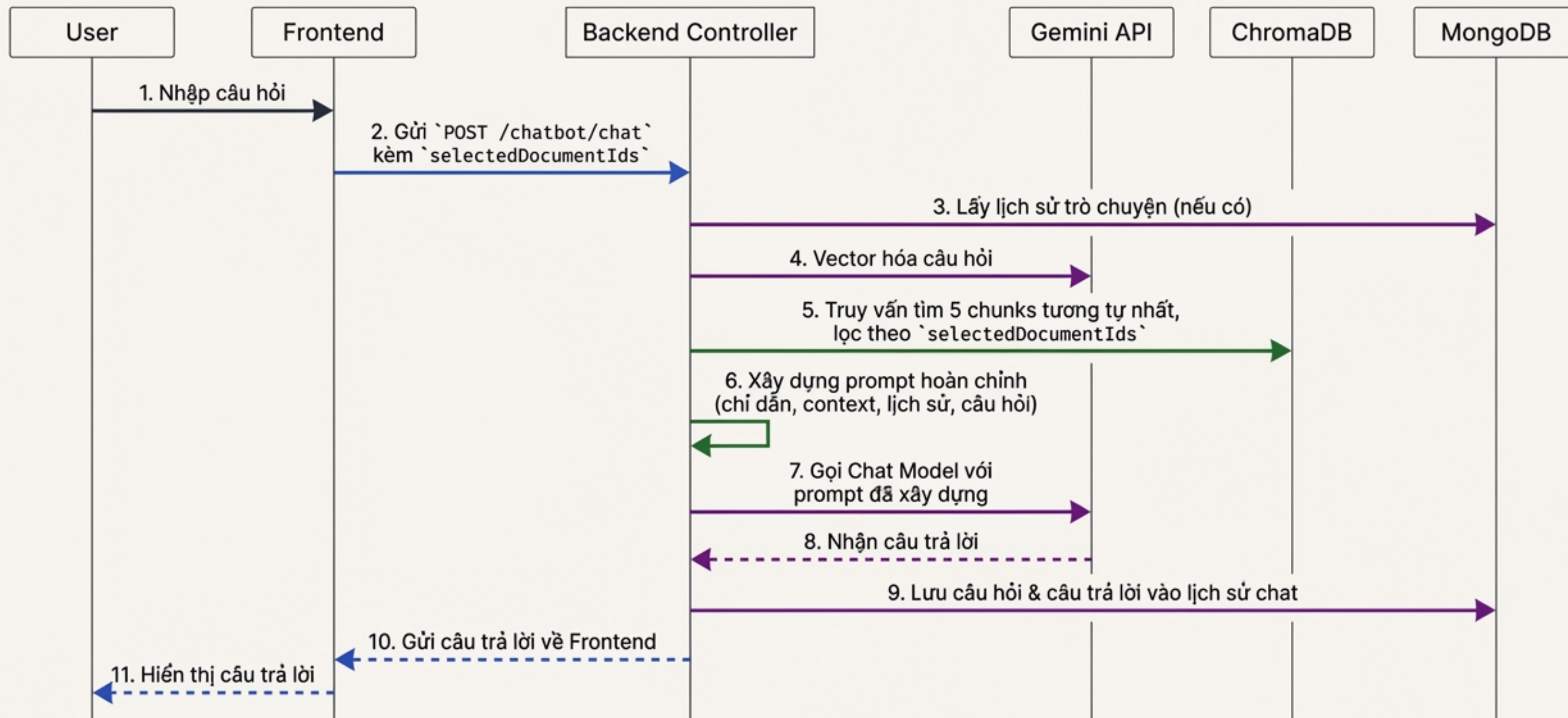
Các chunk liên quan được ghép lại để tạo thành một prompt hoàn chỉnh.



5. Sinh Câu Trả Lời

LLM (gemma-3-4b-it) tạo ra câu trả lời dựa trên ngữ cảnh đã được cung cấp.

Luồng xử lý chi tiết: Chat với cơ chế RAG



Ba bước cốt lõi trong quy trình truy vấn RAG

1. Tìm kiếm Tương đồng Vector

Sử dụng vector của câu hỏi để truy vấn ChromaDB. Lọc kết quả theo các tài liệu đã được người dùng chọn.

```
const results = await collection.query({
  queryEmbeddings: [queryVector],
  nResults: 5,
  where: {
    documentId: { $in: selectedDocumentIds }
  }
});
```

2. Xây dựng Ngữ cảnh

Tạo ra một system prompt lớn, hướng dẫn AI trả lời dựa trên các chunks văn bản tìm được.

```
const systemPrompt = `
Bạn là một trợ lý AI hữu ích.
Hãy trả lời câu hỏi của người dùng dựa trên ngữ cảnh sau:
---
${contextChunks.join('\n---\n')}
---
Câu hỏi: ${userQuestion}
`;
```

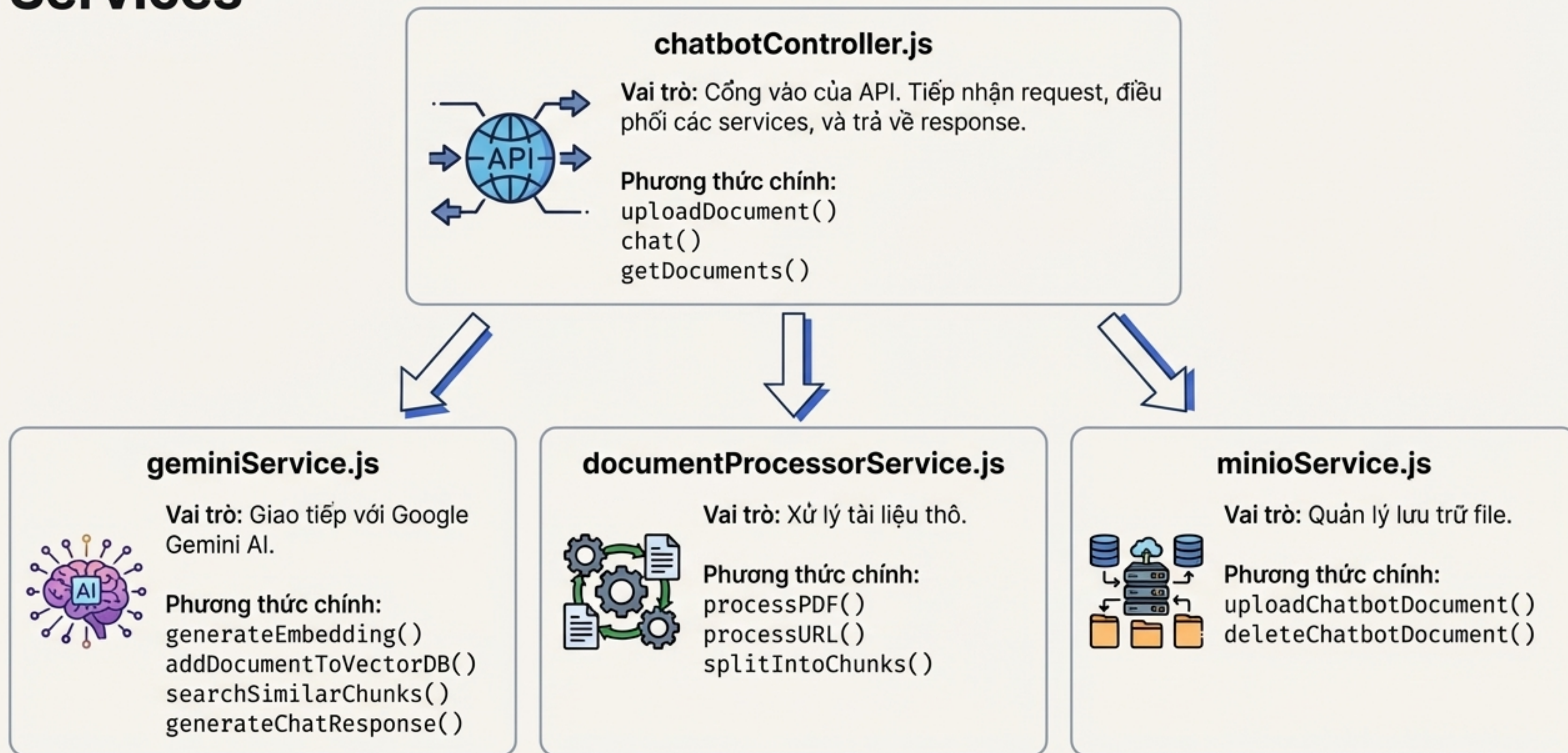
3. Sinh Câu trả lời

Khởi tạo một phiên chat với Gemini API, cung cấp lịch sử trò chuyện và prompt prompt hệ thống đã xây dựng.

```
const chat = geminiModel.startChat({
  history: chatHistory,
  systemInstruction: systemPrompt,
});

const result = await chat.sendMessage(userQuestion);
```


Cấu trúc Backend: Phân chia trách nhiệm giữa các Services



Mô hình dữ liệu: Cách hệ thống lưu trữ và quản lý thông tin

MongoDB Collections

`KnowledgeBase` (ChatbotDocument)

Mục đích: Theo dõi trạng thái xử lý và thông tin quản trị của từng tài liệu.

```
_id
userId
title
status processing ready
fileUrl
chromaCollectionId
```

`ChatHistory` (ChatSession)

Mục đích: Duy trì ngữ cảnh hội thoại và truy vết nguồn thông tin.

```
sessionId
userId
▼ messages
  > { role: "user", content: "..."}
  > { role: "assistant", content: "..."}
knowledgeBaseSources
```

ChromaDB Collections

Cấu trúc Collection: `kb\_user\_{userId}`

Mục đích: Lưu trữ các vector để thực hiện tìm kiếm tương đồng ngữ nghĩa tốc độ cao.

ChromaDB Document Structure

id
doc456_chunk_0

embedding
[0.12, 0.45, ...]



document
This is the first chunk of text...

metadata
documentId: "doc456"
chunkIndex: 0

Giao diện lập trình ứng dụng (API Endpoints)

Quản lý Knowledge Base

POST /api/chatbot/knowledge-base/upload

Tải lên tài liệu PDF hoặc từ URL.

GET /api/chatbot/knowledge-base/documents

Lấy danh sách tất cả tài liệu.

DELETE /api/chatbot/knowledge-base/documents/:id

Xóa một tài liệu.

Tương tác Chat

POST /api/chatbot/chat

Gửi tin nhắn và nhận phản hồi RAG.

```
{ "message", "sessionId", "selectedDocumentIds" }
```

GET /api/chatbot/chat/history/:sessionId

Lấy lịch sử của một phiên chat.

DELETE /api/chatbot/chat/sessions/:sessionId

Xóa một phiên chat.

Sẵn sàng cho thực tế: Tối ưu hiệu suất và các thực hành tốt nhất



Tối ưu hóa hiệu suất

- **Xử lý song song:** `Promise.all` để tạo embedding cho nhiều chunk cùng lúc.
- **Xử lý nền:** Trả về response ngay sau khi upload, xử lý nặng ở background.
- **Chiến lược Chunking:** Kích thước 1000 ký tự với 200 ký tự chồng lấp (overlap) để giữ ngữ cảnh.



Các vấn đề thường gặp

- **"Processing failed"**: Lỗi do file PDF, quota API, hoặc kết nối DB.
- **"No relevant context found"**: Query không khớp. Giải pháp: điều chỉnh chunk, tăng `topK`.
- **"ChromaDB connection refused"**: Service ChromaDB chưa chạy.

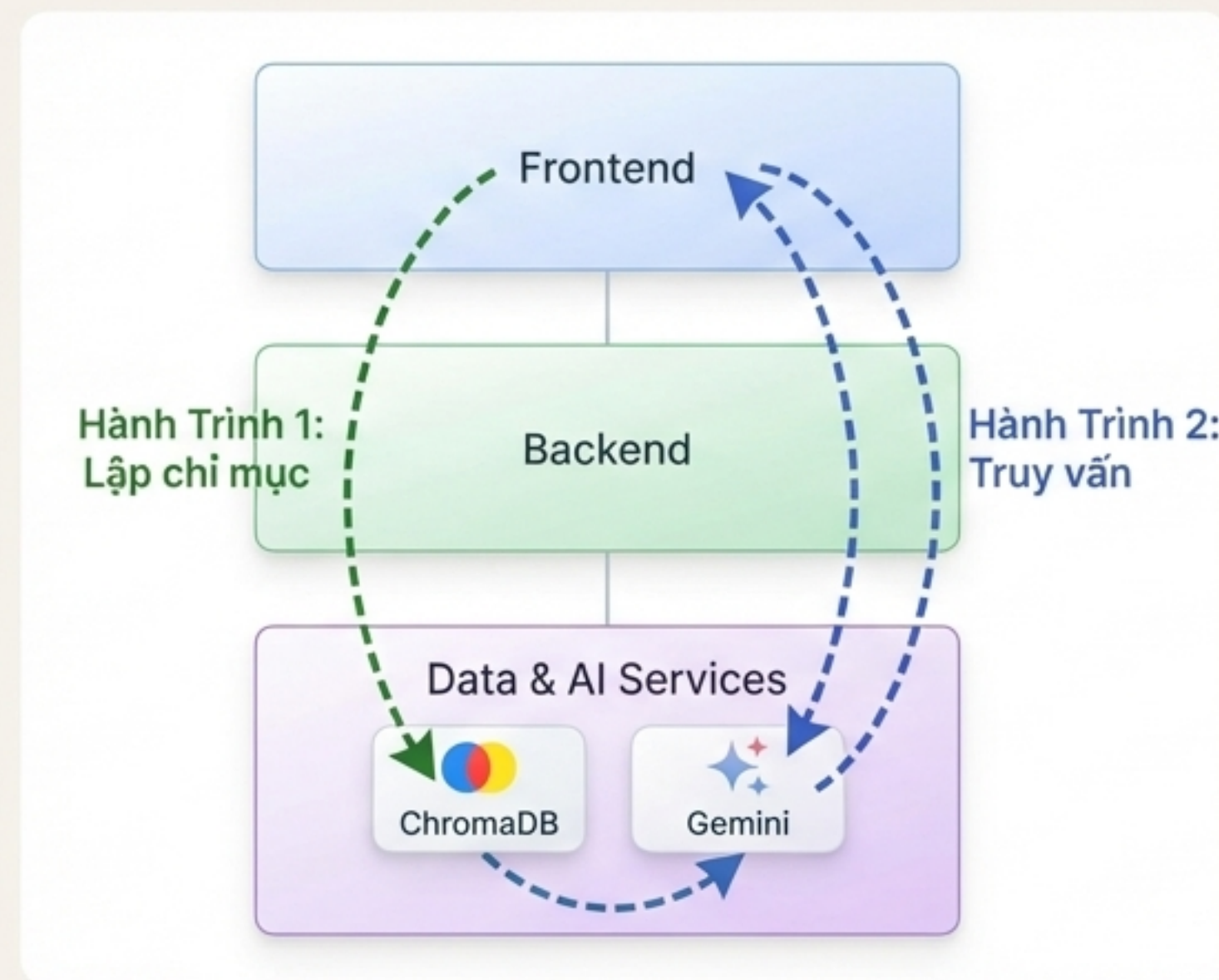


Thực hành tốt nhất

- **Prompt Engineering:** Hướng dẫn hệ thống rõ ràng, chỉ dùng 3-5 chunk liên quan nhất.
- **Bảo mật:** Giới hạn loại file, kích thước file, và cách ly dữ liệu người dùng (`separate collections`).
- **Xử lý lỗi:** Có phương án dự phòng (fallback) khi không tìm thấy ngữ cảnh.

Tổng kết: Bức tranh toàn cảnh của hệ thống Chatbot RAG

- **Vấn đề & Giải pháp:** Chúng ta đã hiểu tại sao RAG là một giải pháp hiệu quả để khắc phục các hạn chế của LLM, tăng độ chính xác và khả năng tùy chỉnh kiến thức.
- **Kiến trúc 3 lớp:** Hệ thống có một cấu trúc rõ ràng với **Frontend**, **Backend**, và **lớp Dữ liệu & AI**, mỗi lớp có một vai trò chuyên biệt.
- **Hành trình của Dữ liệu:** Chúng ta đã theo dõi hai luồng xử lý chính:
 1. **Lập chỉ mục:** Biến một tài liệu PDF thành các vector tri thức trong ChromaDB.
 2. **Truy vấn:** Chuyển một câu hỏi của người dùng thành một câu trả lời thông minh, có ngữ cảnh.
- **Nền tảng vững chắc:** Hệ thống được xây dựng với các mô hình dữ liệu, API và các thực hành tốt nhất để đảm bảo hiệu suất và khả năng mở rộng.



Tài liệu tham khảo và các bước tiếp theo

Tài liệu tham khảo chính



Google Gemini API Docs:
<https://ai.google.dev/docs>



ChromaDB Documentation:
<https://docs.trychroma.com/>



RAG Pattern Guide (Pinecone):
<https://www.pinecone.io/learn/retrieval-augmented-generation/>

Các cấu hình chính

Embedding Model: `text-embedding-004` (768 dimensions)

Chat Model: `gemma-3-4b-it`

Vector DB: ChromaDB with `cosine` similarity

[Link to Project Repository / Internal Documentation](#)